

2026aj05009

August 31, 2026

Work Integrated Learning Programmes Division

M. Tech. in Artificial Intelligence & Machine Learning

1st Semester 2026-27

Assignment I

AIMLZC416 — Mathematical Foundations for Machine Learning

2026aj05009 - Saurabh Tayal *Stayal*

Q1) Finding solutions of linear systems 1. Write a code taking as input a matrix A of size $m \times n$ and a vector b of size $m \times 1$, where m and n are arbitrarily large numbers and $m < n$, constructing the augmented matrix and performing - REF, and - RREF without using any built-in functions. In case you encounter any division by 0, you can choose a different A and/or b . Note that this part should be explicitly there in the code and give a comment line on the same. (1 mark + 1 mark)

```
[ ]: import numpy as np

# Q1(1): Define reusable functions to calculate REF and RREF of [A | b].
# A has m rows and n coefficient columns, and b has m rows and one column.
# Appending b to A creates the augmented matrix [A | b] with shape m x (n + 1).
# The row operations below are written explicitly to show the elimination logic.
# This is a manual Gaussian-elimination approach, which is expected in the ↵
↵assignment.
# It helps us understand how pivoting and elimination preserve the solution set ↵
↵while converting the matrix into a simpler triangular form.

def matrix_multiply(first_matrix, second_matrix):
    # If the first matrix is p x q and the second is q x r, their product is p ↵
    ↵x r. Each output entry is the sum of q corresponding products.
    # This function is used repeatedly throughout the notebook to perform ↵
    ↵matrix multiplication without built-in functions.
    first_rows = len(first_matrix)
    first_columns = len(first_matrix[0])
    second_rows = len(second_matrix)
    second_columns = len(second_matrix[0])

    if first_columns != second_rows:
```

```

        raise ValueError("The matrices cannot be multiplied: inner dimensions
↪differ.")

```

```

product = np.zeros((first_rows, second_columns))
for i in range(first_rows):
    for j in range(second_columns):
        total = 0.0
        for k in range(first_columns):
            total += first_matrix[i, k] * second_matrix[k, j]
        product[i, j] = total
return product

```

```

def transpose_matrix(matrix):
    # The transpose changes rows into columns. The original entry at (i, j) is
↪copied to position (j, i) in the transposed matrix.
    # This is useful in later tasks where we compute  $C = X^T X$  and project
↪vectors onto eigen-directions.
    rows = len(matrix)
    columns = len(matrix[0])
    transposed = np.zeros((columns, rows))
    for i in range(rows):
        for j in range(columns):
            transposed[j, i] = matrix[i, j]
    return transposed

```

```

def ref_form(augmented_matrix, tolerance=1e-12):
    # Use a floating-point copy so row operations do not modify the input.
    # REF (Row Echelon Form) keeps all entries below each pivot equal to zero.
    # This is the first stage of Gaussian elimination and is needed to identify
↪pivot positions.
    ref_matrix = augmented_matrix.copy().astype(float)
    number_of_rows, number_of_columns = ref_matrix.shape
    coefficient_columns = number_of_columns - 1
    pivot_row = 0

    # Scan coefficient columns from left to right. 'pivot_row' means the next
↪row where a successful pivot must be placed; it changes only after a pivot.
    for column in range(coefficient_columns):
        # If all rows already contain pivots, no further elimination is needed.
        if pivot_row == number_of_rows:
            break

        # Search the current row and all rows below it for a non-zero entry.
        # Treat very small floating-point values as zero using the tolerance.
        candidate_row = pivot_row

```

```

while (
    candidate_row < number_of_rows
    and abs(ref_matrix[candidate_row, column]) < tolerance
):
    candidate_row += 1

    # If no usable entry is found, the column has no pivot. Skip it instead
    ↪ of dividing by zero; keep pivot_row unchanged for the next column.
    if candidate_row == number_of_rows:
        continue

    # Move the usable row into the next pivot position. Row swaps preserve
    ↪ the solution set while placing a non-zero pivot where it is needed.
    ref_matrix[[pivot_row, candidate_row]] = ref_matrix[[candidate_row,
    ↪ pivot_row]]

    # The pivot is non-zero because of the search above, so division is
    ↪ safe.
    # Scale the complete row so that its pivot entry becomes exactly 1.
    ref_matrix[pivot_row] = ref_matrix[pivot_row] / ref_matrix[pivot_row,
    ↪ column]

    # Subtract a multiple of the pivot row from each lower row. This makes
    ↪ every entry below the pivot zero and creates the REF structure.
    for next_row in range(pivot_row + 1, number_of_rows):
        multiplier = ref_matrix[next_row, column]
        ref_matrix[next_row] = (
            ref_matrix[next_row] - multiplier * ref_matrix[pivot_row]
        )

    # A pivot was successfully created, so the next pivot belongs in the
    ↪ next row. If a column had no pivot, this increment would not occur.
    pivot_row += 1

    return ref_matrix

def rref_form(augmented_matrix, tolerance=1e-12):
    # REF has zeros below each pivot. Begin with REF and remove entries above
    ↪ each pivot to obtain Reduced Row Echelon Form.
    # RREF makes the solution structure obvious because each pivot column
    ↪ contains only one non-zero entry, equal to 1.
    rref_matrix = ref_form(augmented_matrix, tolerance)
    number_of_rows, number_of_columns = rref_matrix.shape
    coefficient_columns = number_of_columns - 1

```

```

    # Process rows from bottom to top. This ensures that eliminating an entry
    ↪above a pivot does not disturb pivots that have already been handled below.
    for pivot_row in range(number_of_rows - 1, -1, -1):
        # The first non-zero coefficient in this row identifies its pivot
        ↪column.
        pivot_column = 0
        while (
            pivot_column < coefficient_columns
            and abs(rref_matrix[pivot_row, pivot_column]) < tolerance
        ):
            pivot_column += 1

        # If no coefficient is non-zero, this is an all-zero row with no pivot.
        if pivot_column == coefficient_columns:
            continue

        # The pivot was normalized to 1 during REF. Eliminate its entries from
        ↪every row above so the pivot is the only non-zero value in its column.
        for previous_row in range(pivot_row):
            multiplier = rref_matrix[previous_row, pivot_column]
            rref_matrix[previous_row] = (
                rref_matrix[previous_row] - multiplier * rref_matrix[pivot_row]
            )

        return rref_matrix

print("Assuming the size of A = m x n, where m < n, as 4 x 5 for this example.")
m = 4
n = 5

# Generate reproducible random decimal entries as required by the assignment.
# We intentionally choose m < n so the system is underdetermined and therefore
    ↪has free variables.
SEED = 2026
np.random.seed(SEED)
A = np.random.uniform(0.1, 9.9, size=(m, n))
# Generate an independent random right-hand-side vector b.
# This is the target vector we want to solve for: A x = b.
b = np.random.uniform(0.1, 9.9, size=(m, 1))
augmented = np.hstack((A, b))

print("\nMatrix A =")
print(A)
print("\nVector b =")
print(b)
print("\nAugmented matrix [A | b] =")

```

```

print(augmented)

# Compute and display REF and RREF. The functions are reused in Q1.2 and Q1.3.
# These outputs are the symbolic evidence that the row operations preserve the
↪solution set.
ref = ref_form(augmented)
rref = rref_form(augmented)

print("\nREF =")
print(ref)
print("\nRREF =")
print(rref)

```

Assuming the size of $A = m \times n$, where $m < n$, as 4×5 for this example.

Matrix $A =$

```

[[2.24958722 4.14751502 9.67102769 0.97121041 4.79704062]
 [9.77799484 2.04178267 9.03480878 5.57085113 7.77564879]
 [3.06427058 3.99354974 3.00047968 8.86206692 0.57211601]
 [4.4616629 5.60656011 3.66440338 6.90603218 9.1490072 ]]

```

Vector $b =$

```

[[0.39981206]
 [1.2897733 ]
 [2.37268242]
 [6.40039011]]

```

Augmented matrix $[A \mid b] =$

```

[[2.24958722 4.14751502 9.67102769 0.97121041 4.79704062 0.39981206]
 [9.77799484 2.04178267 9.03480878 5.57085113 7.77564879 1.2897733 ]
 [3.06427058 3.99354974 3.00047968 8.86206692 0.57211601 2.37268242]
 [4.4616629 5.60656011 3.66440338 6.90603218 9.1490072 6.40039011]]

```

REF =

```

[[ 1.          1.84367824  4.29902321  0.43172828  2.13240926  0.17772685]
 [-0.          1.          2.06440951 -0.08441387  0.81792121  0.0280275 ]
 [-0.         -0.          1.         -1.09550634  0.68219089 -0.27752693]
 [-0.         -0.         -0.          1.         -1.37331156 -0.4552538 ]]

```

RREF =

```

[[ 1.          0.          0.          0.          1.83636994  0.77608257]
 [ 0.          1.          0.          0.          2.39951861  1.59211701]
 [-0.         -0.          1.          0.         -0.82228063 -0.77626035]
 [-0.         -0.         -0.          1.         -1.37331156 -0.4552538 ]]

```

- Write a Python code to identify the pivot and non-pivot columns and find the particular solution and solutions to $Ax = 0$. (1 mark)

```

[ ]: # Q1(2): Continue with A, b, rref_form, and matrix_multiply from part 1.1.
# This cell also defines the consistency check needed before solving  $Ax = b$ .
# The RREF identifies pivot variables and free variables.
# The logic is: first reduce the system, then classify variables into pivot/
↳ free, then construct one solution and the full family of solutions.

def has_inconsistent_row(rref_matrix, coefficient_columns, tolerance=1e-12):
    # In the RREF of  $[A \mid b]$ , a row of the form  $[0 \ 0 \ \dots \ 0 \mid c]$ , where  $c$  is
    ↳ non-zero, represents  $0 = c$ .
    # This contradiction means that the system  $Ax = b$  has no solution.
    rows = len(rref_matrix)
    for row in range(rows):
        coefficients_are_zero = True

        # Check whether every coefficient in this row is effectively zero.
        # The tolerance handles tiny round-off values produced by
        ↳ floating-point arithmetic.
        for column in range(coefficient_columns):
            if abs(rref_matrix[row, column]) >= tolerance:
                coefficients_are_zero = False
                break

        # If all coefficients are zero but the augmented value is non-zero, the
        ↳ row is inconsistent and the function immediately reports True.
        if (
            coefficients_are_zero
            and abs(rref_matrix[row, coefficient_columns]) >= tolerance
        ):
            return True

    # No contradictory row was found, so the system is consistent.
    return False

# Recalculate RREF from the same augmented matrix created in part 1.1.
rref = rref_form(augmented)

# A consistency test must happen before extracting solutions. If the system is
↳ inconsistent, no particular solution or general solution can be formed.
if has_inconsistent_row(rref, n):
    print("The system is inconsistent and has no solution.")
    raise SystemExit("Program stopped because no solution exists.")

# Identify pivot columns by scanning each RREF row from left to right.
# The first non-zero coefficient in a non-zero row is that row's pivot.

```

```

# A pivot column corresponds to a dependent variable that is solved directly by
↳ the system.
pivot_columns = []
for row in range(m):
    for column in range(n):
        if abs(rref[row, column]) > 1e-12:
            pivot_columns.append(column)
            break

# Every coefficient column that is not a pivot column represents a free
↳ variable.
# These variables can be assigned arbitrary values.
non_pivot_columns = []
for column in range(n):
    if column not in pivot_columns:
        non_pivot_columns.append(column)

print("Pivot columns (0-based):", pivot_columns)
print("Non-pivot columns (0-based):", non_pivot_columns)

# Choose zero for every free variable. Reading the augmented column then gives
↳ one valid assignment for the pivot variables, called a particular solution.
# This is the special solution  $x_p$  that satisfies  $A x_p = b$ .
particular_solution = np.zeros(n)
for row, pivot_column in enumerate(pivot_columns):
    # 'row' identifies the RREF equation, while 'pivot_column' identifies the
↳ actual variable solved by that equation. They need not have the same value.
    particular_solution[pivot_column] = rref[row, n]

print("\nParticular solution  $x_p$  =")
print(particular_solution)
print("Check  $A x_p$  =")
# Reshape  $x_p$  into an  $n \times 1$  column vector because  $A$  has shape  $m \times n$ .
# The manual multiplication returns an  $m \times 1$  vector, which is flattened for
↳ display.
print(matrix_multiply(A, particular_solution.reshape(n, 1)).reshape(m))
print("Check  $b$  =")
print(b.reshape(m))

# Construct homogeneous solutions for  $Ax = 0$ . Process one free variable at a
↳ time:
# set the selected free variable to 1 and all other free variables to 0.
# Each such vector is a basis vector for the null space of  $A$ .
null_space_vectors = []
for free_column in non_pivot_columns:
    vector = np.zeros(n)

```

```

vector[free_column] = 1

    # For each RREF equation, solve the pivot variable in terms of the selected
    ↪ free variable. The minus sign comes from moving the free-variable term to
    ↪ the other side of the equation.
    for row, pivot_column in enumerate(pivot_columns):
        vector[pivot_column] = -rref[row, free_column]
    null_space_vectors.append(vector)

print("\nVectors for solutions of A x = 0:")
for number, vector in enumerate(null_space_vectors, start=1):
    print("v" + str(number) + " =", vector)
    print("A v" + str(number) + " =")
    # Each result should be the zero vector, apart from insignificant
    ↪ floating-point round-off errors such as 1e-15.
    print(matrix_multiply(A, vector.reshape(n, 1)).reshape(m))

# The complete solution of Ax = b is one particular solution plus any linear
    ↪ combination of the homogeneous (null-space) solutions.
# This is the standard structure of solutions to a consistent linear system
    ↪ with free variables.
print("\nGeneral solution:")
print("x = x_p + c1*v1 + c2*v2 + ...")
print("Actual general solution:")
print(
    "x = " + np.array2string(particular_solution, precision=6, separator=", "),
    ↪ end=""
)
for i, vector in enumerate(null_space_vectors, start=1):
    print(
        " + c" + str(i) + "*" + np.array2string(vector, precision=6,
        ↪ separator=", "),
        end="",
    )
print()

# Verification step: choose random values for the free parameters and
    ↪ substitute them into the general solution.
# If the theory is correct, A * x_general should still equal b because each
    ↪ null-space vector satisfies A * v_i = 0.
print("\nVerification of the general solution:")
print("This is the verification step using random values of the free parameters.
    ↪ ")
random_coefficients = np.random.uniform(-2, 2, size=len(null_space_vectors))
x_general = particular_solution.copy()
for coefficient, vector in zip(random_coefficients, null_space_vectors):

```

```

x_general = x_general + coefficient * vector

print("Random coefficients c:", random_coefficients)
print("x_general =")
print(x_general)
print("A x_general =")
print(matrix_multiply(A, x_general.reshape(n, 1)).reshape(m))
print("b =")
print(b.reshape(m))
print(
    "This verifies the general solution because A x_general = b for the chosen_
    random coefficients."
)

```

Pivot columns (0-based): [0, 1, 2, 3]

Non-pivot columns (0-based): [4]

Particular solution $x_p =$

[0.77608257 1.59211701 -0.77626035 -0.4552538 0.]

Check $A x_p =$

[0.39981206 1.2897733 2.37268242 6.40039011]

Check $b =$

[0.39981206 1.2897733 2.37268242 6.40039011]

Vectors for solutions of $A x = 0$:

$v_1 = [-1.83636994 -2.39951861 0.82228063 1.37331156 1.]$

$A v_1 =$

[-8.88178420e-16 6.21724894e-15 -1.99840144e-15 0.00000000e+00]

General solution:

$x = x_p + c_1 v_1 + c_2 v_2 + \dots$

Actual general solution:

$x = [0.776083, 1.592117, -0.77626, -0.455254, 0.] + c_1 [-1.83637, -2.399519, 0.822281, 1.373312, 1.]$

Verification of the general solution:

This is the verification step using random values of the free parameters.

Random coefficients c : [-1.0562656]

$x_{\text{general}} =$

[2.71577695 4.12664597 -1.64480709 -1.90583555 -1.0562656]

$A x_{\text{general}} =$

[0.39981206 1.2897733 2.37268242 6.40039011]

$b =$

[0.39981206 1.2897733 2.37268242 6.40039011]

This verifies the general solution because $A x_{\text{general}} = b$ for the chosen random coefficients.

3. Consider a random 5×7 matrix A and a suitable b and show the REF, RREF, pivot columns,

non-pivot columns, the particular solution, the solutions to $Ax = 0$, the general solution and verify the general solution. ($1/4 \times 8 = 2$ marks)

```
[ ]: # Q1(3): Continue using ref_form, rref_form, matrix_multiply, and
      ↪ has_inconsistent_row from parts 1.1 and 1.2.
      # This cell applies the complete solution process to the required 5 x 7 example.
      # A 5 x 7 system has 5 equations and 7 variables, so a full-row-rank example
      ↪ normally has 5 pivot variables and 2 free variables.
      # This is the general pattern for a rectangular system with more variables than
      ↪ equations.

      np.random.seed(SEED)
      A_5x7 = np.random.uniform(0.1, 9.9, size=(5, 7))
      # Generate b independently, as required. The solution will be obtained later
      # from the RREF; it is not supplied in advance.
      # This ensures we are solving a genuine system rather than assuming a solution
      ↪ beforehand.
      b_5x7 = np.random.uniform(0.1, 9.9, size=(5, 1))
      augmented_5x7 = np.hstack((A_5x7, b_5x7))

      # Reuse the functions from Q1.1. Row reduction transforms the augmented system
      ↪ but preserves its solution set.
      ref_5x7 = ref_form(augmented_5x7)
      rref_5x7 = rref_form(augmented_5x7)

      print("A =")
      print(A_5x7)
      print("\nb =")
      print(b_5x7.reshape(5))
      print("\nREF of [A | b] =")
      print(ref_5x7)
      print("\nRREF of [A | b] =")
      print(rref_5x7)

      # Before solving, test for a row [0 0 ... 0 | c] with c non-zero.
      # Such a row represents the contradiction 0 = c and proves that no solution
      ↪ exists.
      if has_inconsistent_row(rref_5x7, 7):
          print("\nThe 5 x 7 system is inconsistent and has no solution.")
          raise SystemExit("Program stopped because no solution exists.")

      # Scan only the seven coefficient columns. The final column is b and must not
      ↪ be treated as a variable or as a pivot column.
      # This identifies which variables are pivot variables and which are free
      ↪ variables.
      pivot_columns_5x7 = []
      for row in range(5):
```

```

for column in range(7):
    if abs(rref_5x7[row, column]) > 1e-12:
        pivot_columns_5x7.append(column)
        break

# Columns without pivots correspond to free variables. Their values determine
↳ the family of solutions because there are more variables than equations.
non_pivot_columns_5x7 = [
    column for column in range(7) if column not in pivot_columns_5x7
]

print("\nPivot columns:", pivot_columns_5x7)
print("Non-pivot columns:", non_pivot_columns_5x7)

# Set all free variables to zero. The augmented value in each pivot row then
↳ gives one particular solution of  $Ax = b$ .
# This special solution is one member of the complete solution set.
particular_solution_5x7 = np.zeros(7)
for row, pivot_column in enumerate(pivot_columns_5x7):
    particular_solution_5x7[pivot_column] = rref_5x7[row, 7]

print("\nParticular solution x_p =")
print(particular_solution_5x7)
print("Check A x_p =")
print(matrix_multiply(A_5x7, particular_solution_5x7.reshape(7, 1)).reshape(5))
print("Check b =")
print(b_5x7.reshape(5))

# Build one vector for the null space for each free variable.
# For each vector, one free variable is set to 1 and the other free variables
↳ are set to 0. The RREF equations determine all pivot-variable values.
# These vectors form a basis for the null space of A: they satisfy  $A v = 0$ .
null_space_vectors_5x7 = []
for free_column in non_pivot_columns_5x7:
    vector = np.zeros(7)
    vector[free_column] = 1
    for row, pivot_column in enumerate(pivot_columns_5x7):
        vector[pivot_column] = -rref_5x7[row, free_column]
    null_space_vectors_5x7.append(vector)

print("\nSolutions to  $A x = 0$ :")
for number, vector in enumerate(null_space_vectors_5x7, start=1):
    print("v" + str(number) + " =", vector)
    print("A v" + str(number) + " =")
    # These products should be zero, with tiny numerical round-off values
    ↳ allowed.
    print(matrix_multiply(A_5x7, vector.reshape(7, 1)).reshape(5))

```

```

# Since this example has two free variables, every solution is described by two
↳ arbitrary constants multiplying the two null-space basis vectors.
# The general solution is  $x = x_p + c_1 v_1 + c_2 v_2$ .
print("\nGeneral solution:")
print("x = x_p + c1*v1 + c2*v2")
print("Actual general solution:")
print(
    "x = " + np.array2string(particular_solution_5x7, precision=6, separator=",",
↳ ),
    end="",
)
for i, vector in enumerate(null_space_vectors_5x7, start=1):
    print(
        " + c" + str(i) + "*" + np.array2string(vector, precision=6,
↳ separator=", "),
        end="",
    )
print()

# Verification step: choose random values for the free parameters and check
↳ that the full general solution still satisfies  $Ax = b$ .
# This confirms the theory: adding a null-space vector does not change the
↳ equation because  $A v = 0$ .
print("\nVerification of the general solution:")
print("This is the verification step using random values of the free parameters.
↳ ")
random_coefficients_5x7 = np.random.uniform(-2, 2,
↳ size=len(null_space_vectors_5x7))
x_general_5x7 = particular_solution_5x7.copy()
for coefficient, vector in zip(random_coefficients_5x7, null_space_vectors_5x7):
    x_general_5x7 = x_general_5x7 + coefficient * vector

print("Random coefficients c:", random_coefficients_5x7)
print("x_general =")
print(x_general_5x7)
print("A x_general =")
print(matrix_multiply(A_5x7, x_general_5x7.reshape(7, 1)).reshape(5))
print("b =")
print(b_5x7.reshape(5))
print(
    "This verifies the general solution because  $A x_{\text{general}} = b$  for the chosen
↳ random coefficients."
)
print("The checks above verify  $A x_p = b$  and  $A v_i = 0$ .")

```

A =

```
[2.24958722 4.14751502 9.67102769 0.97121041 4.79704062 9.77799484
 2.04178267]
[9.03480878 5.57085113 7.77564879 3.06427058 3.99354974 3.00047968
 8.86206692]
[0.57211601 4.4616629 5.60656011 3.66440338 6.90603218 9.1490072
 0.39981206]
[1.2897733 2.37268242 6.40039011 2.41214929 9.57528454 1.5618379
 0.55958511]
[4.71042124 6.78925934 8.71795496 6.79261965 5.19880891 4.67665447
 2.62252588]]
```

b =

```
[5.07867715 5.72980397 4.90339048 2.130364 4.51865889]
```

REF of [A | b] =

```
[[ 1. 1.84367824 4.29902321 0.43172828 2.13240926 4.34657289
 0.90762548 2.25760402]
 [-0. 1. 2.80209282 0.07543564 1.37757251 3.27156512
 -0.0596986 1.32298834]
 [-0. -0. 1. -0.49386493 -0.15514792 0.70062275
 -0.01311544 0.1399308 ]
 [ 0. 0. 0. 1. 3.04839611 -2.02878805
 -0.26249639 -0.39215322]
 [-0. -0. -0. -0. 1. 0.17396077
 0.15579005 0.23018352]]
```

RREF of [A | b] =

```
[[ 1. 0. 0. 0. 0. -1.33034018
 0.99075339 -0.08380277]
 [ 0. 1. 0. 0. 0. 4.72754021
 0.77080223 2.10996141]
 [ 0. 0. 1. 0. 0. -0.5362321
 -0.353124 -0.36456778]
 [ 0. 0. 0. 1. 0. -2.55908938
 -0.73740619 -1.09384376]
 [-0. -0. -0. -0. 1. 0.17396077
 0.15579005 0.23018352]]
```

Pivot columns: [0, 1, 2, 3, 4]

Non-pivot columns: [5, 6]

Particular solution x_p =

```
[-0.08380277 2.10996141 -0.36456778 -1.09384376 0.23018352 0.
 0. ]
```

Check A x_p =

```
[5.07867715 5.72980397 4.90339048 2.130364 4.51865889]
```

Check b =

```
[5.07867715 5.72980397 4.90339048 2.130364 4.51865889]
```

Solutions to $Ax = 0$:

```
v1 = [ 1.33034018 -4.72754021  0.5362321   2.55908938 -0.17396077  1.
      0.          ]
```

$A v1 =$

```
[-5.32907052e-15 -2.26485497e-14 -1.77635684e-15 -4.44089210e-16
 -7.99360578e-15]
```

```
v2 = [-0.99075339 -0.77080223  0.353124    0.73740619 -0.15579005  0.
      1.          ]
```

$A v2 =$

```
[-4.44089210e-16  1.77635684e-15 -3.33066907e-16  3.33066907e-16
 -1.77635684e-15]
```

General solution:

$x = x_p + c1*v1 + c2*v2$

Actual general solution:

```
x = [-0.083803,  2.109961, -0.364568, -1.093844,  0.230184,  0.
      0.          ] + c1*[ 1.33034 , -4.72754 ,  0.536232,  2.559089, -0.173961,  1.
      ,
      0.          ] + c2*[-0.990753, -0.770802,  0.353124,  0.737406, -0.15579 ,  0.
      ,
      1.          ]
```

Verification of the general solution:

This is the verification step using random values of the free parameters.

Random coefficients c : [1.75971544 -0.7529851]

$x_{\text{general}} =$

```
[ 3.00323993 -5.6287615   0.31315103  2.85416948  0.04136965  1.75971544
 -0.7529851 ]
```

$A x_{\text{general}} =$

```
[5.07867715 5.72980397 4.90339048 2.130364   4.51865889]
```

$b =$

```
[5.07867715 5.72980397 4.90339048 2.130364   4.51865889]
```

This verifies the general solution because $A x_{\text{general}} = b$ for the chosen random coefficients.

The checks above verify $A x_p = b$ and $A v_i = 0$.

Q2) Consider a dataset $X \in \mathbb{R}^{500 \times 6}$ constructed as follows: the first four features $f1, f2, f3, f4$ are generated as random features sampled from a standard normal distribution (read about this). The fifth and sixth features are defined by the relations $f5 = 2f1 + 3f2$, $f6 = f3 - 2f4$. Perform the following tasks in sequence: 1. Write a Python code to generate the dataset $X = [f1, f2, f3, f4, f5, f6]$. [0.5]

```
[ ]: # Q2(1): Generate the dataset X = [f1, f2, f3, f4, f5, f6].
      # Each of the 500 rows represents one observation and each column represents
      # one feature.
      # The first four features are sampled independently from N(0, 1), the standard
      # normal distribution.
```

```

# Features f5 and f6 are then deliberately created as linear combinations: f5 =
    ↪ 2f1 + 3f2 and f6 = f3 - 2f4.
# These exact relationships will later explain why the six-column matrix has
    ↪ rank 4 rather than rank 6.
# In other words, the last two columns do not add independent information; they
    ↪ are combinations of the first four columns.

import numpy as np

# Fixing the seed makes the experiment reproducible: every execution generates
    ↪ the same X.
np.random.seed(SEED)
number_of_data_points = 500

# randn generates independent samples from a standard normal distribution with
    ↪ mean 0 and variance 1.
# Keeping these four columns independent provides four base directions.
f1 = np.random.randn(number_of_data_points)
f2 = np.random.randn(number_of_data_points)
f3 = np.random.randn(number_of_data_points)
f4 = np.random.randn(number_of_data_points)

# These columns contain no new independent information because they are
    ↪ determined exactly by the first four columns.
# Their dependence is the key structure being studied in Q2.
f5 = 2 * f1 + 3 * f2
f6 = f3 - 2 * f4

# column_stack places the six equal-length feature arrays side by side. Thus X
    ↪ has shape (500, 6), and its fifth and sixth columns satisfy the specified
    ↪ relationships exactly.
X = np.column_stack((f1, f2, f3, f4, f5, f6))

print("Shape of X:", X.shape)
print("First five rows of X:")
print(X[:5])

```

Shape of X: (500, 6)

First five rows of X:

```

[[-0.43171852 -0.70255252 -0.10353308  0.87732991 -2.9710946  -1.85819289]
 [-1.39287397  1.50152579 -0.73937348  1.6569189   1.71882945 -4.05321129]
 [ 0.31157067 -0.60182461  1.21508673  1.27213227 -1.1823325  -1.32917782]
 [-0.01323488  1.44440069  1.47784444  1.3170826   4.3067323  -1.15632076]
 [ 1.44970773 -0.29466117 -0.19591321  0.71605898  2.01543195 -1.62803116]]

```

2. Write a Python code which computes the rank of X and display the output for the dataset X generated in step 1. [0.5]

```
[ ]: # Q2(2): Compute rank(X) without calling a built-in rank function.
# The rank is the number of linearly independent columns, which is equal to the
    ↳ number of pivots produced by Gaussian elimination.
# Since f5 and f6 are exact combinations of f1-f4, the expected rank is 4. Row
    ↳ reduction verifies this numerically.
# The purpose of this task is to show the difference between the matrix
    ↳ dimension (6 columns) and the true number of independent directions (4).

# Work on a floating-point copy so row operations do not change the original
    ↳ dataset X.
reduced_X = X.copy().astype(float)
number_of_rows, number_of_columns = reduced_X.shape
rank_of_X = 0
tolerance = 1e-12

# Examine columns from left to right. rank_of_X is both the number of pivots
    ↳ found and the row where the next successful pivot must be placed.
for column in range(number_of_columns):
    # Once every row has a pivot, no additional independent direction can be
    ↳ found.
    if rank_of_X == number_of_rows:
        break

    # Search the current pivot row and all rows below it for a usable non-zero
    ↳ entry.
    # A tolerance treats tiny floating-point round-off values as mathematical
    ↳ zeros.
    pivot_row = rank_of_X
    while pivot_row < number_of_rows and abs(reduced_X[pivot_row, column]) <
    ↳ tolerance:
        pivot_row += 1

    # A zero remainder in this column means that the column contributes no
    ↳ pivot.
    # Continuing preserves the current pivot row for the next candidate column.
    if pivot_row == number_of_rows:
        continue

    # Swap the usable row into position, then normalize it so the pivot equals
    ↳ 1.
    # These elementary row operations preserve the rank and simplify
    ↳ elimination.
    reduced_X[[rank_of_X, pivot_row]] = reduced_X[[pivot_row, rank_of_X]]
    reduced_X[rank_of_X] = reduced_X[rank_of_X] / reduced_X[rank_of_X, column]
```

```

    # Eliminate entries below the pivot. A successful pivot represents one
    ↪ independent row/column direction, so advance to the next pivot row after
    ↪ elimination.
    for next_row in range(rank_of_X + 1, number_of_rows):
        multiplier = reduced_X[next_row, column]
        reduced_X[next_row] = reduced_X[next_row] - multiplier *
    ↪ reduced_X[rank_of_X]
        rank_of_X += 1

print("Rank of X:", rank_of_X)

```

Rank of X: 4

3. Numerical Experiment with the Power Method

Read about the power method for finding the dominant eigenvalue and its corresponding eigenvector and perform the following tasks.

(a) Write a Python code to compute the covariance matrix [0.5]

$C = (1/n)XTX$ where n is number of data points

```

[ ]: # Q2(3a): Compute the covariance/Gram matrix  $C = (1/n) X^T X$  explicitly.
# The functions transpose_matrix and matrix_multiply are already defined in
    ↪ previous cells and are being reused here.
# Keeping a single shared definition avoids duplicate function redefinitions
    ↪ while preserving the same manual linear-algebra logic.
# This matrix captures pairwise similarities between features; because the data
    ↪ are centered in this formulation, it is the covariance-like matrix used for
    ↪ eigen-analysis.

number_of_data_points = X.shape[0]
transposed_X = transpose_matrix(X)
# Multiplying  $X^T$  by  $X$  accumulates pairwise feature products; division by  $n$ 
    ↪ converts each accumulated sum into the required average over the 500
    ↪ observations.
# Since the matrix is symmetric,  $C = C^T$ , and its eigenvectors are orthogonal.
    ↪ This matters for the later power method and deflation steps.
C = matrix_multiply(transposed_X, X) / number_of_data_points

print("n =", number_of_data_points)
print("Shape of  $X^T$  =", transposed_X.shape)
print("Covariance matrix C =")
print(C)

```

$n = 500$

Shape of $X^T = (6, 500)$

Covariance matrix C =

```

[[ 9.99410950e-01 -1.60051587e-02  1.89588472e-02  9.93510963e-03
   1.95080642e+00 -9.11372085e-04]

```

```

[-1.60051587e-02  1.11925929e+00 -1.33242656e-01  6.06364469e-02
 3.32576755e+00 -2.54515550e-01]
[ 1.89588472e-02 -1.33242656e-01  1.14325526e+00 -3.01520021e-02
-3.61810275e-01  1.20355926e+00]
[ 9.93510963e-03  6.06364469e-02 -3.01520021e-02  9.94448097e-01
 2.01779560e-01 -2.01904820e+00]
[ 1.95080642e+00  3.32576755e+00 -3.61810275e-01  2.01779560e-01
 1.38789155e+01 -7.65369395e-01]
[-9.11372085e-04 -2.54515550e-01  1.20355926e+00 -2.01904820e+00
-7.65369395e-01  5.24165565e+00]]

```

(b) Implement the Power Method in Python to approximate the largest eigenvalue λ_1 and its corresponding eigenvector v_1 of C . Show the code and the outputs. [1]

```

[7]: # Q2(3b): Compute all eigenvalues and eigenvectors from the covariance matrix,
      # using the power method with deflation.
      # The procedure is: find the dominant eigenpair of the current matrix, then
      # deflate it and repeat.
      # This gives all eigenpairs of a real symmetric matrix.
      # The logic is based on the fact that for a symmetric matrix, applying the
      # power method repeatedly causes the vector to align with the largest
      # eigenvector, and the Rayleigh quotient then approximates the corresponding
      # eigenvalue.

np.set_printoptions(precision=8, suppress=False)

def vector_norm(vector):
    # Euclidean norm of a vector. The relation  $\|x\| = \sqrt{\sum x_i^2}$  is
    # required before normalization.
    flattened_vector = vector.reshape(-1)
    squared_length = 0.0
    for value in flattened_vector:
        squared_length += value * value
    return np.sqrt(squared_length)

def power_method(matrix, tolerance=1e-7, maximum_iterations=10000):
    # Power method iteration for the largest eigenvalue/eigenvector of a matrix.
    # Step 1: start with a non-zero vector.
    # Step 2: repeatedly multiply by A and normalize.
    # Step 3: use the Rayleigh quotient to approximate the eigenvalue.
    # Step 4: stop when the eigenvalue changes by less than tolerance.
    vector = np.ones((matrix.shape[0], 1))
    vector = vector / vector_norm(vector)
    old_eigenvalue = 0.0

    for iteration in range(1, maximum_iterations + 1):

```

```

new_vector = matrix_multiply(matrix, vector)
new_vector_length = vector_norm(new_vector)
if new_vector_length == 0:
    raise ValueError("The power method reached a zero vector.")
vector = new_vector / new_vector_length

transpose_vector = transpose_matrix(vector)
matrix_vector = matrix_multiply(matrix, vector)
rayleigh_quotient = matrix_multiply(transpose_vector, matrix_vector)
eigenvalue = rayleigh_quotient[0, 0]

if abs(eigenvalue - old_eigenvalue) < tolerance:
    return eigenvalue, vector.reshape(matrix.shape[0]), iteration
old_eigenvalue = eigenvalue

return eigenvalue, vector.reshape(matrix.shape[0]), maximum_iterations

def power_method_all(matrix):
    # Repeatedly apply the power method to the current matrix after deflation.
    # Deflation uses the identity  $A_k = A - \sum_{i=1}^{k-1} \lambda_i v_i v_i^T$ ,
    ↪ which removes the already-found dominant eigen-direction from the matrix.
    current_matrix = matrix.copy().astype(float)
    eigenvalues = []
    eigenvectors = []

    for _ in range(matrix.shape[0]):
        eigenvalue, eigenvector, _ = power_method(current_matrix)
        eigenvalues.append(eigenvalue)
        eigenvectors.append(eigenvector)

        v_column = eigenvector.reshape(matrix.shape[0], 1)
        projector = matrix_multiply(v_column, transpose_matrix(v_column))
        current_matrix = current_matrix - eigenvalue * projector

    eigenvalues = np.array(eigenvalues)
    eigenvectors = np.column_stack(eigenvectors)
    order = np.argsort(eigenvalues)[::-1]
    return eigenvalues[order], eigenvectors[:, order]

power_eigenvalues, power_eigenvectors = power_method_all(C)

print("All eigenvalues from the power method (largest to smallest):")
print(np.array2string(power_eigenvalues, precision=8, separator=", "))
print("\nAll eigenvectors from the power method (columns):")
print(np.array2string(power_eigenvectors, precision=8, separator=", "))

```

```

lambda_1 = power_eigenvalues[0]
v1 = power_eigenvectors[:, 0]
print("\nLargest eigenvalue:", f"{lambda_1:.8f}")
print("Corresponding eigenvector:")
print(np.array2string(v1, precision=8, separator=", "))

```

All eigenvalues from the power method (largest to smallest):

```

[ 1.50422544e+01,  6.21210776e+00,  1.12396155e+00,  9.98621091e-01,
 -6.29432747e-24, -3.61254503e-08]

```

All eigenvectors from the power method (columns):

```

[[ 0.13287204,  0.03618485,  0.49901362, -0.66816109,  0.49644621,
   -0.49644621],
 [ 0.23092481,  0.00815514, -0.32586398,  0.4445457 , -0.15892044,
    0.15892044],
 [-0.03490847,  0.21055279,  0.70893898,  0.53387526,  0.59427535,
   -0.59427535],
 [ 0.02798583, -0.34845616,  0.37442392,  0.26628587,  0.3250008 ,
   -0.3250008 ],
 [ 0.95851852,  0.09683514,  0.0204353 , -0.00268508,  0.51613112,
   -0.51613112],
 [-0.09088013,  0.90746511, -0.03990886,  0.00130353, -0.05572623,
    0.05572623]]

```

Largest eigenvalue: 15.04225443

Corresponding eigenvector:

```

[ 0.13287204,  0.23092481, -0.03490847,  0.02798583,  0.95851852,
 -0.09088013]

```

- (c) Write a Python code to obtain the next largest eigenvalue λ_2 and its corresponding eigenvector v_2 by applying power method on $C - \lambda_1 v_1 v_1^T$. Having found out $v_1, v_2, \dots, v_{k-1}$, one can find λ_k and its corresponding eigenvector v_k by applying power method on $C - \sum_{j=1}^{k-1} \lambda_j v_j v_j^T$. Give the code and also display the obtained eigenvalues and the corresponding eigenvectors. [1.5]

```

[8]: # Q2(3c): Obtain the next eigenpair by deflating the dominant direction and
      # then applying the power method again.
      # The matrix used for the second eigenpair is the deflated matrix  $C - \lambda_1 v_1 v_1^T$ ,
      # which avoids repeating the dominant eigenpair.
      # This follows the general deflation rule: after finding eigenpairs  $(\lambda_1, v_1), \dots, (\lambda_k, v_k)$ ,
      # the next eigenpair is found by applying the power method to  $A - \sum_{j=1}^k \lambda_j v_j v_j^T$ .
      # This is the numerical way of extracting the next largest eigen-direction
      # while preserving orthogonality.

v1_column = v1.reshape(C.shape[0], 1)
projector_on_v1 = matrix_multiply(v1_column, transpose_matrix(v1_column))

```

```

deflated_from_v1 = C - lambda_1 * projector_on_v1

lambda_2, v2, iterations_2 = power_method(deflated_from_v1)

# print("Deflated matrix for the second eigenpair:")
# print(np.array2string(deflated_from_v1, precision=8, separator=", "))
print("\nSecond eigenvalue:", f"{lambda_2:.8f}")
print("Corresponding eigenvector:")
print(np.array2string(v2, precision=8, separator=", "))
print("Number of iterations for the second eigenpair:", iterations_2)

print("\nTop two eigenpairs from the power method:")
print("(1, v1) =", f"{lambda_1:.8f}", np.array2string(v1, precision=8,
↪separator=", "))
print("(2, v2) =", f"{lambda_2:.8f}", np.array2string(v2, precision=8,
↪separator=", "))

```

Second eigenvalue: 6.21210776

Corresponding eigenvector:

[0.03618485, 0.00815514, 0.21055279, -0.34845616, 0.09683514,
0.90746511]

Number of iterations for the second eigenpair: 7

Top two eigenpairs from the power method:

(1, v1) = 15.04225443 [0.13287204, 0.23092481, -0.03490847, 0.02798583,
0.95851852,
-0.09088013]

(2, v2) = 6.21210776 [0.03618485, 0.00815514, 0.21055279, -0.34845616,
0.09683514,
0.90746511]

- (d) Find all the eigenvalues and eigenvectors using Python function and compare with the obtained result in (c). [0.5]

```

[ ]: # Q2(3d): Use NumPy's symmetric eigensolver as a reference for the power-method
↪eigenpairs.
# Only the eigenvalues and eigenvectors are shown; the difference values are
↪not printed.
# This is a comparison step, not a new method. We want to verify whether the
↪power method has correctly recovered the same eigenpairs.
# NumPy returns eigenvalues in ascending order and stores the matching
↪eigenvectors as columns.
eigenvalues_np, eigenvectors_np = np.linalg.eigh(C)

# Sort from largest to smallest to match the power-method presentation.
order = np.argsort(eigenvalues_np)[::-1]
eigenvalues_np = eigenvalues_np[order]

```

```

eigenvectors_np = eigenvectors_np[:, order]

# Align signs only so the eigenvectors can be compared visually.
# Since eigenvectors are determined up to sign, flipping the sign does not
  ↪ change the actual eigen-direction.
for i in range(C.shape[0]):
    if np.dot(power_eigenvectors[:, i], eigenvectors_np[:, i]) < 0:
        power_eigenvectors[:, i] = -power_eigenvectors[:, i]

print("Power-method eigenvalues (largest to smallest):")
print(np.array2string(power_eigenvalues, precision=8, separator=", "))
print("\nNumPy eigenvalues (largest to smallest):")
print(np.array2string(eigenvalues_np, precision=8, separator=", "))

print("\nPower-method eigenvectors (columns):")
print(np.array2string(power_eigenvectors, precision=8, separator=", "))
print("\nNumPy eigenvectors (columns):")
print(np.array2string(eigenvectors_np, precision=8, separator=", "))

```

Power-method eigenvalues (largest to smallest):

```
[ 1.50422544e+01,  6.21210776e+00,  1.12396155e+00,  9.98621091e-01,
 -6.29432747e-24, -3.61254503e-08]
```

NumPy eigenvalues (largest to smallest):

```
[1.50422544e+01, 6.21210772e+00, 1.12396187e+00, 9.98620729e-01,
 3.77522452e-15, 5.75461957e-16]
```

Power-method eigenvectors (columns):

```
[[-0.13287204,  0.03618485, -0.49901362,  0.66816109, -0.49644621,
 -0.49644621],
 [-0.23092481,  0.00815514,  0.32586398, -0.4445457 ,  0.15892044,
  0.15892044],
 [ 0.03490847,  0.21055279, -0.70893898, -0.53387526, -0.59427535,
 -0.59427535],
 [-0.02798583, -0.34845616, -0.37442392, -0.26628587, -0.3250008 ,
 -0.3250008 ],
 [-0.95851852,  0.09683514, -0.0204353 ,  0.00268508, -0.51613112,
 -0.51613112],
 [ 0.09088013,  0.90746511,  0.03990886, -0.00130353,  0.05572623,
  0.05572623]]
```

NumPy eigenvectors (columns):

```
[[-0.13287044,  0.03619598, -0.5000847 ,  0.6672603 ,  0.53448354,
  0.00645224],
 [-0.23092445,  0.00818196,  0.32657507, -0.44395781,  0.80172531,
  0.00967836],
 [ 0.03491781,  0.210542 , -0.7080928 , -0.53515488, -0.00492798,
  0.40821855],
```

```
[-0.02800128, -0.34845685, -0.37398023, -0.26695668, 0.00985596,
 -0.81643709],
[-0.95851422, 0.09693785, -0.02044418, 0.00264718, -0.26724177,
 -0.00322612],
[ 0.09092036, 0.9074557 , 0.03986767, -0.00124151, 0.00492798,
 -0.40821855]]
```

- (e) Compare the number of iterations required to get an accuracy of 10^{-7} using the power method. The actual values can be taken as the one obtained in (d). [0.5]

```
[ ]: # Q2(3e): Compare the number of iterations needed by the power method to reach
      ↪  $1e-7$  accuracy for every eigenvalue.
      # The NumPy eigenvalues are already sorted from largest to smallest in the
      ↪ previous comparison step.
      # The fix is to preserve that ordering and deflate only the already-processed
      ↪ larger eigen-directions.
      # This is the iteration-count experiment: we test how many repetitions are
      ↪ needed for each eigen-direction to converge to the required tolerance.

def count_iterations_for_target(
    matrix, actual_eigenvalue, tolerance=1e-7, max_iterations=10000
):
    # This helper computes how many power-method iterations are needed to
    ↪ approximate a target eigenvalue within a specified error.
    # It starts from a unit vector, repeatedly applies the matrix, normalizes,
    ↪ and computes the Rayleigh quotient.
    vector = np.ones((matrix.shape[0], 1))
    vector = vector / vector_norm(vector)

    for iteration in range(1, max_iterations + 1):
        new_vector = matrix_multiply(matrix, vector)
        new_vector_length = vector_norm(new_vector)
        if new_vector_length == 0:
            raise ValueError("The power method reached a zero vector.")
        vector = new_vector / new_vector_length

        transpose_vector = transpose_matrix(vector)
        matrix_vector = matrix_multiply(matrix, vector)
        rayleigh_quotient = matrix_multiply(transpose_vector, matrix_vector)
        approximate_eigenvalue = rayleigh_quotient[0, 0]
        error = abs(approximate_eigenvalue - actual_eigenvalue)

        if error < tolerance:
            return iteration

    return max_iterations
```

```

# Keep the exact eigenvalues in the already-correct descending order from the
↳ NumPy comparison cell.
# Reversing them again causes the smaller eigenvalues to be checked first and
↳ makes the counts look wrong.
exact_eigenvalues_desc = eigenvalues_np.copy()
exact_eigenvectors_desc = eigenvectors_np.copy()

all_iterations = []
for mode_index in range(len(exact_eigenvalues_desc)):
    target_eigenvalue = exact_eigenvalues_desc[mode_index]

    # Deflate the already-processed larger eigen-directions only.
    # This ensures the next iteration focuses on the current eigenvalue rather
    ↳ than re-approximating a previously found dominant direction.
    deflated_matrix = C.copy().astype(float)
    for previous_index in range(mode_index):
        vector = exact_eigenvectors_desc[:, previous_index].reshape(C.shape[0],
↳ 1)
        projector = matrix_multiply(vector, transpose_matrix(vector))
        deflated_matrix = (
            deflated_matrix - exact_eigenvalues_desc[previous_index] * projector
        )

        iterations_needed = count_iterations_for_target(deflated_matrix,
↳ target_eigenvalue)
        all_iterations.append(iterations_needed)

print("Required accuracy:", 1e-7)
# print("Exact eigenvalues from NumPy (largest to smallest):")
# print(np.array2string(exact_eigenvalues_desc, precision=8, separator=", "))
print("\nIterations needed for each eigenvalue to reach 1e-7 accuracy:")
print(all_iterations)

```

Required accuracy: 1e-07

Iterations needed for each eigenvalue to reach 1e-7 accuracy:

[11, 6, 53, 1, 1, 1]